

Cómo escribimos una base de datos

Notación para nombrar tablas y columnas, y cómo pasar del dibujo a las tablas

Material introductorio

Rel Guzman



**Universidad
Tecnológica
del Perú**

Logro de la sesión

Al finalizar, sabrás **escribir una tabla** con nombres y tipos correctos, y **convertir un dibujo de entidades** en tablas conectadas entre sí.

¿Por qué acordar una forma de nombrar?

El mismo dato, escrito por cuatro personas

<code>nombreCliente</code>	<code>NombreDelCliente</code>	<code>nom_cli</code>	<code>nombre</code>
----------------------------	-------------------------------	----------------------	---------------------

El problema no es la apariencia, es la memoria

Los cuatro guardan lo mismo. Pero quien escriba una consulta mañana tiene que **adivinar** cuál se usó. Acordar una sola forma evita esa adivinanza.

En este curso usaremos **una sola convención**, la misma en todas las sesiones.

Tres palabras para leer una tabla

id	nombre	correo
1	Raúl	raul@correo.com
2	Mariana	mariana@correo.com

Tabla	Columna	Fila
Todo el cuadro. Guarda datos de un mismo asunto. <i>Ejemplo:</i> cliente.	Un tipo de dato que anotamos de todos. <i>Ejemplo:</i> correo.	Un caso concreto, leído de izquierda a derecha. <i>Ejemplo:</i> "2, Mariana".

Del dibujo a la tabla: cada cosa tiene su equivalente

En el dibujo (diseño conceptual)	En la tabla (diseño lógico)	Ejemplo en la tienda
Entidad · aquello de lo que guardamos datos	Tabla	cliente
Atributo · un dato de esa entidad	Columna	correo
Instancia · un caso concreto	Fila	"2, Mariana"
Identificador · el dato que no se repite nunca	Clave primaria	id
Relación · el vínculo entre dos entidades	Clave foránea	un cliente hace pedidos

Cómo usar este cuadro

Primero se dibuja: entidades, sus atributos y las relaciones entre ellas. Después se traduce: cada entidad pasa a ser una tabla. **Las tres primeras filas las acabas de ver**; el identificador aparece en unas diapositivas más, y las relaciones son el tema de la segunda parte.

Tipo de dato

Aviso a la base sobre qué clase de valor va en esa columna: número, texto, fecha...

Para qué sirve

Si `precio` es un número, la base puede sumarlo. Si fuera texto, “89” + “35” daría “8935”.

Y también protege

Si `fecha_registro` es una fecha, la base rechaza “ayer por la tarde”. El dato entra bien o no entra.

Tipo	Guarda	Ejemplo de valor
INT	números enteros	42
VARCHAR(80)	texto de hasta 80 caracteres	'Mariana'
DECIMAL(10,2)	números con 2 decimales	89.00
DATE	una fecha	'2026-03-15'
BOOLEAN	verdadero o falso	TRUE

Ojo con esto

Para dinero usamos `DECIMAL(10,2)` y nunca un número con decimales aproximados: 10 dígitos en total, 2 después del punto. Así los centavos salen exactos.

```
CREATE TABLE cliente (  
  nombre          VARCHAR(80)  
  apellido        VARCHAR(80)  
  correo          VARCHAR(120)  
  fecha_registro  DATE  
);
```

Cómo se lee

“Crea una tabla llamada `cliente` con estas cuatro columnas, y cada una guarda esto”.

Lo mínimo

Un nombre de tabla, y por cada columna un nombre y un tipo. Esta tabla ya funciona; en las próximas dos diapositivas la mejoramos.

Restricción

Regla escrita junto a una columna. La base rechaza cualquier dato que la incumpla.

Restricción	Qué obliga	Ejemplo en cliente
NOT NULL	La columna no puede quedar vacía	Un cliente sin nombre no debería existir
UNIQUE	El valor no se puede repetir	Dos clientes no pueden tener el mismo correo
DEFAULT	Valor que se pone solo si no indicas otro	activo empieza en TRUE

Clave primaria

Columna que identifica a cada fila y nunca repite su valor.

Ejemplo

En `cliente`, el `id 1` es Raúl y siempre lo será.
Nunca habrá dos filas con `id 1`.

Aunque haya dos clientes llamados Raúl, sus
`id` son distintos.

Con la clave primaria y las restricciones, la tabla `cliente` queda completa. La vemos en la siguiente diapositiva.

AUTO_INCREMENT

La base pone el siguiente número sola: 1, 2, 3...
Tú no lo escribes al insertar un cliente nuevo.

```
CREATE TABLE cliente (  
  id            INT            PRIMARY KEY AUTO_INCREMENT,  
  nombre        VARCHAR(80)    NOT NULL,  
  apellido      VARCHAR(80)    NOT NULL,  
  correo        VARCHAR(120)   UNIQUE,  
  fecha_registro DATE,  
  activo        BOOLEAN        DEFAULT TRUE  
);
```

Cómo se lee de corrido: tabla cliente; su id es un entero que la base numera sola y sirve de clave primaria; el nombre y el apellido son obligatorios; el correo no puede repetirse; la fecha de registro es una fecha; y todo cliente nuevo empieza como activo.

Regla	Así sí	Así no
La tabla va en singular y en minúscula	cliente	Clientes
Si el nombre de la tabla lleva varias palabras, se unen con guion bajo	metodo_pago	MetodosDePago
En las columnas se hace igual	fecha_registro	fechaRegistro
La clave primaria siempre se llama id	id	id_cliente
Sin tildes ni ñ en los nombres	anio	año

Falta una regla más

Algunas tablas tendrán columnas que apuntan a **otra** tabla. Cómo se llaman esas columnas lo veremos en la segunda parte, cuando aparezca la razón por la que existen.

Pasos:

1. Abre Workbench: *File* → *New Model* y doble clic en *Add Diagram*.
2. Arrastra el ícono de tabla al lienzo y nómbrala cliente.
3. Escribe las seis columnas con sus tipos, copiando la diapositiva 11.
4. Marca las casillas y escribe TRUE en *Default* de activo.
5. Guarda el modelo y toma una captura.

Las casillas de Workbench:

Casilla	Es la restricción
PK	Clave primaria
AI	AUTO_INCREMENT
NN	NOT NULL
UQ	UNIQUE

Qué debe verse al terminar

id con PK y AI · nombre y apellido con NN · correo con UQ · activo con TRUE por defecto.

Preguntas



Universidad
Tecnológica
del Perú



Entidad

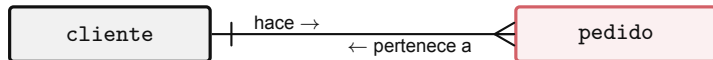
Algo del mundo real del que queremos guardar datos.

Ejemplo: un cliente, un pedido. **Cada entidad será una tabla.**

Relación

El vínculo entre dos entidades, con un **verbo en cada sentido**.

Ejemplo: un cliente **hace** pedidos; un pedido **pertenece a** un cliente.



La marca —|— significa **uno** y la pata de gallo —< significa **muchos**.

Cardinalidad: ¿cuántos hay de cada lado?

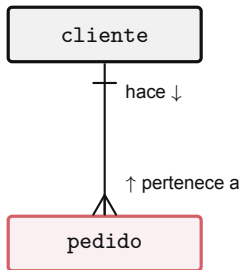
Cardinalidad

Cuántos elementos de una entidad se relacionan con cuántos de la otra.
Se pregunta **dos veces**, una por cada lado.

Nombre	Ejemplo de todos los días	En la tienda
1:N	Una madre tiene varios hijos, pero cada hijo tiene una madre	Un cliente y sus pedidos
N:M	Un alumno lleva varios cursos y un curso tiene varios alumnos	Un pedido y sus productos
1:1	Una persona tiene un DNI y ese DNI es de una sola persona	Un cliente y su clave de acceso

Se lee “uno a muchos”, “muchos a muchos” y “uno a uno”. **La cardinalidad decide cómo quedan las tablas**: veremos los tres casos en ese orden, y de cada uno haremos el diagrama.

Uno a muchos (1:N): aparece la clave foránea



Un cliente hace **muchos** pedidos; cada pedido es de **un solo** cliente.

Clave foránea

Columna que guarda el `id` de otra tabla, para saber a qué fila pertenece.

La regla, y de dónde sale su nombre

La clave foránea va en la tabla del lado de la **pata de gallo**: en `pedido`, no en `cliente`. Y se llama como la tabla a la que apunta más `_id`: `cliente_id`. En los dibujos la marcaremos **FK**, igual que **PK** marca la clave primaria.

1:N con datos: el id del cliente se repite en pedido

cliente	
id	nombre
1	Raúl
2	Mariana

id es la clave primaria

pedido		
id	cliente_id	fecha_pedido
1	1	2026-03-01
2	1	2026-03-05
3	2	2026-03-07

cliente_id es la clave foránea

Léelo en los datos: el cliente **1** aparece en dos filas de `pedido`, así que Raúl hizo dos pedidos. Pero cada fila de `pedido` nombra a **un solo** cliente, porque en esa celda cabe un valor. Ahí está el “uno a muchos”.

Sobre el diagrama que ya tienes:

1. Crea la tabla pedido con id, fecha_pedido y total. **No escribas** cliente_id.
2. En la barra de herramientas elige 1:n
Non-Identifying Relationship.
3. Clic primero en pedido y después en cliente.
4. Mira lo que pasó: Workbench creó cliente_id solo.

El orden de los clics importa

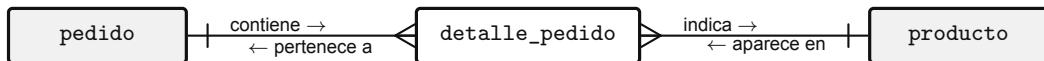
Primero la tabla que **recibe** la clave foránea (el lado muchos), después la tabla a la que **apunta**. Al revés, la columna aparece en cliente y la relación queda invertida.

Qué debe verse al terminar

Una línea entre las dos tablas, y dentro de pedido una columna cliente_id que tú no escribiste, marcada como clave foránea.

Muchos a muchos (N:M): hace falta una tabla en medio

Un pedido lleva **muchos** productos y un producto aparece en **muchos** pedidos.



Por qué no basta una clave foránea

¿En cuál de las dos tablas la pondrías? En **pedido** tendrías que guardar varios **producto_id** en una sola celda, y una celda guarda **un** valor.

La regla

Una **tabla intermedia** —o **tabla asociativa**— con **dos** claves foráneas. Ahí caben además los datos de la combinación, como la cantidad.

pedido	
id	cliente_id
1	1
2	2

producto	
id	nombre
1	Audífonos
2	Cargador

detalle_pedido			
id	pedido_id	producto_id	cantidad
1	1	1	2
2	1	2	1
3	2	1	1

pedido_id y producto_id son las dos claves foráneas

Léelo en los datos: el pedido **1** aparece dos veces (llevó dos productos) y el producto **1** aparece dos veces (estuvo en dos pedidos). Que **ambos** se repitan es lo que hace “muchos a muchos”. La cantidad vive aquí porque solo tiene sentido para esa combinación.

Sobre el mismo diagrama:

1. Crea producto con id, nombre y precio.
2. En la barra de herramientas elige *n:m Identifying Relationship*.
3. Clic en pedido y después en producto.
4. Workbench crea sola una tabla en el medio, con una clave foránea hacia cada lado.
5. Renómbrala detalle_pedido y agrégale cantidad y precio_unitario.

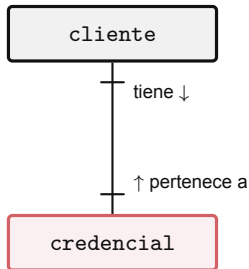
¿Por qué la crea sola?

Porque una relación N:M **no se puede guardar entre dos tablas**: siempre hace falta una tercera. La herramienta hace lo mismo que harías a mano, y no te deja saltarte ese paso.

Dos cosas que hay que corregir a mano

El nombre: la crea como pedido_has_producto, que rompe nuestras reglas de nombrado. **La clave primaria:** Workbench usa la pareja pedido_id + producto_id; en nuestro esquema le agregamos además un id propio.

Uno a uno (1:1): clave foránea que no se puede repetir



Un cliente tiene **una** clave de acceso, y esa clave es de **un solo** cliente: las dos marcas son de **uno**.

La regla

Clave foránea en cualquiera de las dos tablas, pero con **UNIQUE**: así ningún cliente puede tener dos credenciales.

Sin el **UNIQUE** no es 1:1

Nada impediría que el mismo `cliente_id` apareciera en dos filas de `credencial`: la tabla aceptaría 1:N aunque el dibujo diga 1:1.

1:1 con datos: ningún id se repite

cliente	
id	nombre
1	Raúl
2	Mariana

credencial		
id	cliente_id	usuario
1	1	raulg
2	2	marianap

cliente_id es clave foránea y UNIQUE

Léelo en los datos: compara con la tabla pedido de hace unas diapositivas. Allí el cliente_id 1 salía dos veces; aquí no puede salir dos veces. Esa es toda la diferencia entre 1:N y 1:1, y la impone el UNIQUE.

Sobre el mismo diagrama:

1. Crea credencial con id y usuario.
2. Elige *1:1 Non-Identifying Relationship*.
3. Clic primero en credencial y después en cliente.
4. Abre la columna cliente_id que apareció y comprueba que tenga marcada la casilla UQ.

La casilla que hace la diferencia

Si UQ no quedó marcada, márcala tú. Sin ella el dibujo dice 1:1 pero la tabla se comporta como 1:N.

Qué debe verse al terminar

Cinco tablas conectadas: cliente, pedido, producto, detalle_pedido y credencial. Guarda el modelo y toma una captura del diagrama completo.

Coronel, C., Morris, S., & Rob, P. (2011). *Bases de datos: diseño, implementación y administración* (9.^a ed.). Cengage Learning.

Elmasri, R., & Navathe, S. (2007). *Fundamentos de sistemas de bases de datos* (5.^a ed.). Addison Wesley.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database system concepts* (7th ed.). McGraw-Hill.



gracias



Universidad
Tecnológica
del Perú